



AI/ML Interview Q&A Guide

**Cracking Applied AI/ML
Engineering Interviews**



Lamhot Siagian

Cracking Applied AI/ML Engineering Interviews: Interview Q&A Guide

A practical Q&A book for speaking clearly, designing on a whiteboard, and shipping reliable
AI systems

Lamhot Siagian, PhD Student - AI Evaluation Engineer - AI Engineer Machine
Learning - Data Science & AI/ML

January 2026

Copyright

© 2026 Lamhot Siagian. All rights reserved.

Lamhot Siagian, PhD Student - AI Evaluation Engineer - AI Engineer Machine Learning - Data Science & AI/ML

Contact

LinkedIn: <https://www.linkedin.com/in/lamhotsiagian/>

Website: softwaretestarchitect.com

Email: lamhotsiagian2025@gmail.com

Publication month: January 2026

No part of this publication may be reproduced, stored, or transmitted in any form or by any means without prior written permission of the author, except for brief quotations in reviews.

Disclaimer: The examples and scenarios in this book are for educational purposes only. They do not represent any employer, past or present, and they are not production advice for regulated or high-risk systems without proper review.

Contents

1	Agentic AI Systems: Planning, Tools, and State	1
	Q1: What is an agent in applied AI engineering?	1
	Q2: How do you structure an agent loop for reliability?	2
	Q3: Planning vs. reactive agents: what is the tradeoff?	2
	Q4: What is tool calling, and why do strict schemas matter?	2
	Q5: How do you handle memory in an agent?	2
	Q6: What are common agent failure modes?	2
	Q7: How do you make retries safe?	2
	Q8: How do you evaluate an agent beyond “it looks good”?	2
	Q9: How do you debug agent behavior in production?	3
	Q10: What is a minimal production checklist for an agent?	3
2	Tool Standards and Interoperability: MCP and Tool Contracts	5
	Q1: What is a tool contract, and what must it include?	5
	Q2: What problem do standards like MCP-style patterns solve?	6
	Q3: How do you design for least privilege?	6
	Q4: How do you handle authentication and secrets for tools?	6
	Q5: How do you version tool schemas safely?	6
	Q6: What does a good error model look like for tool calls?	6
	Q7: How do you prevent tool misuse by the model?	6
	Q8: What is the impact of latency and timeouts on agent design?	6
	Q9: How do you test tool contracts?	7
	Q10: How do you roll out a breaking tool change without breaking agents?	7
3	LLM Security Engineering: Prompt Injection, Tool Abuse, and Sandboxing	8
	Q1: What is prompt injection in LLM systems?	8
	Q2: What is the “confused deputy” risk with tool calling?	9
	Q3: How do you protect secrets and credentials?	9
	Q4: What role does input and output validation play?	9
	Q5: How do you sandbox risky tools?	9
	Q6: How do you handle untrusted retrieved content in RAG?	9
	Q7: What is a practical set of defenses for tool abuse?	9
	Q8: How do you test LLM security?	9
	Q9: How do you respond to an LLM security incident?	10
	Q10: How do you run LLM security testing for prompt injection and tool abuse?	10

4	RAG 2.0: Hybrid Retrieval, Reranking, and Grounded Answers	11
	Q1: What are the core components of a RAG system?	11
	Q2: Why use hybrid retrieval instead of vectors only?	12
	Q3: What does reranking add, and when is it worth it?	12
	Q4: How do you choose a chunking strategy?	12
	Q5: How do you handle freshness and updates?	12
	Q6: How do permissions work in RAG?	12
	Q7: What does “grounded” output mean operationally?	12
	Q8: How do you detect and reduce hallucinations in RAG?	12
	Q9: What metrics do you use for RAG evaluation?	13
	Q10: How do you implement citations and source attribution in RAG?	13
5	Evaluation-First AI Engineering: Testing LLMs, RAG, and Agents	14
	Q1: What is a good starting point for an evaluation suite?	14
	Q2: How do you design a rubric for LLM answers?	15
	Q3: What metrics matter for RAG evaluation?	15
	Q4: How do you evaluate agents with tool calls?	15
	Q5: What is LLM-as-a-judge and what are the risks?	15
	Q6: Offline vs online evaluation: why do you need both?	15
	Q7: How do you prevent regressions when prompts change?	15
	Q8: How do you build adversarial tests?	15
	Q9: What is a practical failure-bucketing scheme?	16
	Q10: How do you set up an automated evaluation gate for LLM changes?	16
6	LLMOps and Observability: Tracing, Cost, Latency, and Quality	17
	Q1: What do you log for an LLM request in production?	17
	Q2: What is the difference between logs, metrics, and traces?	18
	Q3: How do you define SLOs for LLM applications?	18
	Q4: How do you control prompt and model changes?	18
	Q5: How do you analyze a sudden cost increase?	18
	Q6: What is prompt drift and how do you detect it?	18
	Q7: How do you handle incident response for LLM systems?	18
	Q8: What are common observability pitfalls in LLM systems?	18
	Q9: How do you make replays reproducible?	18
	Q10: What are the most useful dashboards and alerts for LLMOps?	19
7	Inference Engineering: Serving, Caching, and Cost Optimization	20
	Q1: What are the main drivers of LLM inference latency?	20
	Q2: What is prefill vs decode time?	21
	Q3: How does batching help, and what is the risk?	21
	Q4: What is quantization and when do you use it?	21
	Q5: What caching strategies are common in LLM apps?	21
	Q6: How do you route between small and large models?	21
	Q7: What is tail latency and why does it matter?	21
	Q8: How do you reduce context size without losing accuracy?	21
	Q9: What capacity planning inputs do you need for GPU inference?	22
	Q10: When do you use batching, streaming, and caching in inference?	22

8	Small Language Models and Edge AI: Distillation, On-Device, and Routing	23
	Q1: What problems do SLMs solve compared to large models?	23
	Q2: What is distillation and when is it effective?	24
	Q3: How do you decide which tasks to run on-device?	24
	Q4: How do you handle long-tail failures with SLMs?	24
	Q5: What are common optimization techniques for SLM deployment?	24
	Q6: What does hybrid routing look like in practice?	24
	Q7: How do privacy constraints influence architecture?	24
	Q8: What evaluation differences exist for on-device models?	24
	Q9: How do you update on-device models safely?	25
	Q10: How do you route requests between small and large models?	25
9	Multimodal AI: Vision, Audio, and UI-Facing Systems	26
	Q1: What are common components in a multimodal application?	26
	Q2: How do you handle noisy images in production?	27
	Q3: What is the role of OCR in vision-language pipelines?	27
	Q4: How do you evaluate a multimodal system?	27
	Q5: What are common safety and privacy issues with images and audio?	27
	Q6: How do you manage latency for multimodal inputs?	27
	Q7: What is prompt design like for vision-language tasks?	27
	Q8: How do you debug wrong answers in a multimodal system?	27
	Q9: How do you handle domain shift in images?	28
	Q10: How do you evaluate and debug multimodal model outputs?	28
10	Post-Training and Alignment: SFT, LoRA/PEFT, RLHF/RLAIF, and DPO	29
	Q1: What is supervised fine-tuning (SFT)?	29
	Q2: What is PEFT/LoRA and why is it popular?	30
	Q3: What is RLHF and what problem does it solve?	30
	Q4: What is RLAIF?	30
	Q5: What is DPO and when would you use it?	30
	Q6: How do you prevent overfitting during fine-tuning?	30
	Q7: What data mistakes cause the worst failures?	30
	Q8: How do you evaluate post-training results?	30
	Q9: How do you decide between prompting and fine-tuning?	31
	Q10: How do you choose between prompt/RAG, LoRA, and full fine-tuning?	31
11	Synthetic Data and Privacy: Utility, Leakage, and Governance	32
	Q1: What are the main use cases for synthetic data?	32
	Q2: Why can synthetic data leak private information?	33
	Q3: How do you measure the utility of synthetic data?	33
	Q4: What privacy tests do you run?	33
	Q5: How do you generate synthetic data in practice?	33
	Q6: What is the danger of synthetic bias?	33
	Q7: How do you use synthetic data for testing LLM systems?	33
	Q8: How do you combine real and synthetic data in training?	33
	Q9: What governance practices are important?	34
	Q10: How do you test for privacy leakage in synthetic data?	34

12 Modern AI Data Architecture: Lakehouse, Vectors, Graphs, and Streaming Features	36
Q1: What is the difference between a data lake, warehouse, and lakehouse?	37
Q2: What is an online vs offline feature pipeline?	37
Q3: What is training-serving skew and how do you prevent it?	37
Q4: How do you keep a vector index fresh?	37
Q5: When do you use a graph database in AI systems?	37
Q6: What data quality checks are essential for ML pipelines?	37
Q7: How do you manage lineage and reproducibility?	37
Q8: How do you design access control for AI data systems?	38
Q9: What is CDC and why is it useful?	38
Q10: How do you keep vector indexes consistent with source data?	38
13 Terminology	40
References	43
A Interview Drill Plan	45
How to use this plan	45
7-Day Plan	45
Final drill (repeat weekly)	46

Chapter 1

Agentic AI Systems: Planning, Tools, and State

Interview Anchor

- **Agent:** A system that can plan and take actions with tools to reach a goal, not just generate text.
- **Tool call:** A controlled side effect (API, DB query, code execution) issued by the model through a strict interface.
- **Reliability:** The agent does the right action, once, with observable state and recoverable failures.

Whiteboard framework (Agent Loop):

1. Define goal, constraints, and allowed tools.
2. Plan a short sequence of steps with checkpoints.
3. Execute one step, log state, and validate output.
4. Repair or retry with limits; otherwise escalate.
5. Commit result and produce a grounded response.

Example questions:

- Conceptual: What makes an LLM system an “agent” rather than a chatbot?
- Scenario: Your agent double-charges users when retries happen. How do you fix it?

Agentic systems matter because interviews now test more than prompt writing. Teams want assistants that can complete tasks, call tools safely, and recover from failures. In real products, an agent touches user data and external systems, so reliability is the main bar. A strong candidate explains how to constrain actions and observe behavior end-to-end.

In the lifecycle, agents sit on top of retrieval, tools, and runtime controls. You design the tool surface first, then the agent loop, then the evaluation harness. After launch, you monitor for tool errors, unsafe actions, and drift in outcomes. In interviews, this maps to architecture, failure analysis, and operational readiness.

A useful mental model is “a stateful workflow with an LLM in the loop.” The LLM proposes a step, but the system owns state, permissions, and validation. You treat every action as a transaction with logging and idempotency. You also assume the model will sometimes be wrong, so you design for detection and recovery.

Q1: What is an agent in applied AI engineering?

A: An agent is an AI system that can decide and execute actions using tools to reach a goal. A good answer mentions planning, tool invocation, and persistent state. In practice, I separate “thinking”

from “doing” by forcing actions through typed tool APIs. For example, an agent that schedules meetings must call a calendar API with validated parameters and log every attempt.

Q2: How do you structure an agent loop for reliability?

A: I use a short iterative loop: plan, execute one step, validate, and update state. I set strict retry limits and treat every tool call as a transaction. I also store checkpoints so the agent can resume without repeating side effects. For example, “send email” returns a message ID that becomes the idempotency key.

Q3: Planning vs. reactive agents: what is the tradeoff?

A: Planning improves coherence and reduces thrashing, but it can overcommit to a bad plan. Reactive behavior adapts quickly but can loop or waste calls. In interviews, I say I prefer shallow plans with frequent checkpoints. For example, I generate a 3-step plan, then re-plan after each tool result.

Q4: What is tool calling, and why do strict schemas matter?

A: Tool calling is structured invocation of external functions using a constrained interface. Strict schemas reduce ambiguity and prevent prompt injection from turning into arbitrary actions. In practice, I use JSON schemas, enums for sensitive parameters, and allowlists for endpoints. For example, “create_ticket” only allows pre-approved project keys and sanitized summaries.

Q5: How do you handle memory in an agent?

A: I treat memory as data with scope and purpose, not as free-form chat history. I separate short-term working memory (current task state) from long-term preferences (stable user settings). I also version memory writes and require explicit triggers. For example, “remember my timezone” is stored as a structured field, not as a paragraph.

Q6: What are common agent failure modes?

A: The big ones are tool misuse, infinite loops, stale context, and silent hallucination of actions. Another common issue is non-idempotent retries that repeat side effects. I mitigate with step budgets, action validation, and post-action verification. For example, after a payment API call, I query the ledger to confirm the charge exists once.

Q7: How do you make retries safe?

A: I use idempotency keys and make tools return stable identifiers. I also classify errors into retryable (timeouts) and non-retryable (validation failures). The agent should re-ask for missing inputs instead of guessing. For example, if an address is invalid, I stop and request the corrected field rather than retrying.

Q8: How do you evaluate an agent beyond “it looks good”?

A: I measure task success rate, tool error rate, cost, and time-to-completion. I also test safety constraints: “did the agent call a disallowed tool?” and “did it expose secrets?” For scenario

coverage, I build a suite of scripted tasks with expected intermediate steps. For example, I assert that “refund” requires an order lookup before issuing the refund call.

Q9: How do you debug agent behavior in production?

A: I rely on traces that link prompts, tool calls, and outputs across a run. I log the plan, each action, tool inputs/outputs, and the final response with redaction. Then I replay runs with fixed seeds or captured tool outputs to isolate logic errors. For example, I can replay a failed run offline by mocking the CRM API response.

Q10: What is a minimal production checklist for an agent?

A: I ship agents with a short checklist: strict tool allowlists, idempotency keys for every write, and traceable state transitions. I add an evaluation gate that runs a small task suite and blocks regressions in success rate, unsafe calls, and cost. I also ship a kill switch and safe fallback responses for timeouts or policy violations. For example, I require post-action verification for payments and disable write tools if anomaly alerts fire.

Interview Cheatsheet

Key terms list

Idempotency:

Repeating a request produces the same single effect.

Step budget:

A hard cap on tool calls or iterations per run.

Checkpoint:

Saved state that allows resume without repeating actions.

Tool allowlist:

Only approved tools and parameters are callable.

Verification step:

A post-action check that confirms the side effect occurred.

Escalation path:

A defined fallback to a human or safe failure response.

Rapid-fire

- **What makes it an agent?** A: It can take validated actions with tools and track state.
- **What is the main risk?** A: Uncontrolled side effects from wrong or repeated tool calls.
- **How do you control actions?** A: Typed tools, allowlists, and approvals for sensitive steps.
- **How do you debug?** A: End-to-end traces that join prompts, tools, and outcomes.

Tell me about a time... (STAR skeleton)**Situation:**

An internal support agent started looping and spiked API costs after a tool timeout pattern changed.

Task:

I needed to stop the cost bleed and make retries safe without reducing completion rate.

Action:

I added idempotency keys, a step budget, retry classification, and trace-based replay to reproduce the loop.

Result:

Tool calls dropped, duplicate actions went to zero, and task success stayed stable under load.